Lesson 05 · Async, Tasks & Concurrency

Task and await, cancellation, fan-out, async streams, channels and shared state — for an engineer who ships Kotlin coroutines, Spring WebFlux and Node

Contents

1 · Why this is the lesson to slow down on 2
1.1 · The numbers to leave this lesson with 2

2 · The Task model — what await really does 2
2.1 · Concept map — coroutines, Reactor and the event loop → .NET 3
2.2 · One await, step by step — a partner call inside ASP.NET Core 4
2.3 · What the compiler builds, written out by hand 4
2.4 · The async toolbox — six types in every .NET service 5

3 · Cancellation and timeouts — tokens, not scopes 5
3.1 · Two tokens, four outcomes — one partner call 6
3.2 · A per-partner deadline on an injected clock 6
3.3 · Four ways to give up — which ones stop the work 6
3.4 · Proof on a fake clock — WaitAsync gives up, the partner does not 7

4 · Fan-out — 30 partners, one deadline 7
4.1 · The same fan-out — Kotlin coroutines vs C# 8
4.2 · What 30 partners returned 8
4.3 · Waiting for 30 partners 8
4.4 · Results as they arrive — Task.WhenEach 8
4.5 · First good-enough quote, then cancel the rest 9
4.6 · When a partner contract caps concurrent calls 9
4.7 · What Task.WhenAll does — and what Kotlin engineers expect it to do 9

5 · Streams and pipelines — IAsyncEnumerable and Channels 9
5.1 · An event stream — Kotlin Flow vs IAsyncEnumerable 10
5.2 · The notification dispatcher — bounded at every stage 10
5.3 · Bounded channels and a producer that waits 11
5.4 · Router and senders — completion flows downstream 11
5.5 · What the dispatcher delivered 11

6 · Shared state and testable time 12
6.1 · Choosing a primitive for shared state 12
6.2 · Interlocked for counters, Lock for read-compare-write 13
6.3 · One partner call per key, however many callers 13
6.4 · Concurrency primitives next to their JVM counterparts 13
6.5 · A lock you can hold across an await — SemaphoreSlim 14
6.6 · A 250 ms deadline tested in microseconds 14

7 · Traps — and what Visual Basic cannot say 14
7.1 · Sync-over-async on a one-thread context 15
7.2 · Why the deadlock never resolves 15
7.3 · async void — the exception the caller never sees 16
7.4 · A forgotten Task.Run — the failure nobody observes 16
7.5 · The four traps, reproduced 17
7.6 · Consuming an async stream — C# vs Visual Basic 17
7.7 · What the Visual Basic compiler rejects 17
7.8 · Async features across the languages you use 18

8 · Hands-on — run the rating desk in both languages 18
8.1 · The same desk — C# vs Visual Basic, one C# library 18
8.2 · Commands 19
8.3 · What you should see 19
8.4 · What the dispatcher prints 19
8.5 · Lines of code in the lesson-05 samples 20

Page numbers are resolved from the rendered PDF. Code panels are cut from the compiled samples.

1 · Why this is the lesson to slow down on

The transfer map in lesson 01 (The .NET Platform Map) rates this lesson 55% familiar • Estimate **— lower than every lesson except Visual Basic — because C# async reads like Kotlin and TypeScript and behaves like neither.** `async/await` looks exactly like a `suspend` function or a TypeScript `async function`, so an experienced engineer from either world writes code that compiles, passes its unit test, and then deadlocks a UI thread, starves the thread pool under load or loses an exception nobody observed.

You have already built the hard version of this problem. Quoting a motor policy against ~30 external partners means calling them concurrently, giving each its own deadline, tolerating the ones that fail or never answer, and returning the best price before the customer leaves. Sending the result over SMS, email and LINE is a producer/consumer pipeline with back-pressure. This lesson builds both in .NET, so every concept lands on a design you already own.

The model differs from yours in three places, and each has a trap attached. A `Task` is already running when you receive it — hot like a JavaScript Promise or a Kotlin `async {}`, unlike a cold Reactor `Mono`. Cancellation is a token you pass by hand: .NET has no structured concurrency, so no scope cancels children for you. And the code after an `await` resumes wherever the current *SynchronizationContext* says, which is why one library behaves differently in a WinForms app, classic ASP.NET and ASP.NET Core.

Every result here is reproducible. Partner latencies are `TimeProvider` timers, so the tests move a fake clock instead of sleeping, and the console output was captured from real runs. Premiums are illustrative, not a real tariff.

1.1 · The numbers to leave this lesson with

PARTNERS PER QUOTE	FAN-OUT WALL CLOCK	DETERMINISTIC TESTS	TASK.WHENEACH	VB ASYNC GAPS
30	**263 ms**	**11**	**.NET 9**	**10**
one fan-out · measured from the catalog	vs 4,982 ms one by one · captured	fake clock · 0 real sleeps · measured	yields tasks as they complete · verified	compiler errors · measured by compiling

KEY: • Same idea • Renamed • Different • Trap • Verified • Estimate • Measured C# Java/Kotlin runtime architecture

i Before you start

1. **Read lesson 02 (C# Language Essentials), lesson 03 (Types, OOP & Generics) and lesson 04 (Collections, LINQ & Functional C#) first.** This lesson uses `using` declarations, exception filters (`catch … when`), switch expressions, records, lambdas and LINQ without explaining them again.
2. **You will build** `L05.PartnerRates` (C# library: partner fan-out with per-partner deadlines, streaming results, thread-safe statistics, a shared token behind a semaphore) and its xUnit tests on virtual time; `L05.RateDesk` and `L05.VbAsync` (the same console desk in C# and VB); `L05.Notifications` (a bounded Channels pipeline); `L05.AsyncTraps` (four traps, reproduced and contained); and `L05.VbRules.Tests` (VB compiler rules proved by compiling snippets).
3. **Where this lesson stops.** Resilient `HttpClient` calls (retry, circuit breaker) and hosting the dispatcher as a `BackgroundService` are lesson 08 (ASP.NET Core Web APIs); test frameworks in depth are lesson 10 (Testing, TDD & 100% Coverage); VB syntax at large is lesson 06 (VB.NET for Legacy Estates).
4. Facts that change between releases are marked • Verified and link to the official source; judgements are marked • Estimate; numbers computed from the samples or captured from their runs are marked • Measured. Kotlin panels are for comparison and are not compiled.

2 · The Task model — what await really does

A `Task` is a promise object for work already under way, and `await` is the compiler rewriting the rest of your method into a callback on it. An `async` method runs synchronously on its caller's thread until it awaits something that has not finished. There it registers the remainder of the method as a continuation, returns an incomplete `Task` to its caller and gives the thread back. No thread waits for the socket or the timer; when the operation completes, the continuation is scheduled and the method resumes where it stopped (Asynchronous programming with async and await).

Where that continuation runs is the decision your other stacks made differently. Kotlin resumes on the dispatcher you name; Node always resumes on its single event-loop thread. .NET asks the *SynchronizationContext* that was current at the `await`. A WinForms or WPF UI thread and classic ASP.NET on .NET Framework install one, so continuations queue back to that thread. ASP.NET Core, console apps and worker services have none, so continuations run on any thread-pool thread (ConfigureAwait FAQ).

`Task<T>` is the default return type; `ValueTask<T>` is an optimisation with rules. `ValueTask<T>` saves an allocation when a result is usually available at once, such as a cache hit, but it may be awaited only once, and the documented default is to return

`Task<T>` unless performance analysis proves otherwise (ValueTask<TResult>). `void` is for event handlers only, and `IAsyncEnumerable<T>` marks an async stream (async return types).

.NET 11 moves the machinery, not the meaning. From .NET 11 Preview 4 the runtime libraries are compiled with *runtime async*: the runtime, instead of a compiler-generated state machine, suspends and resumes async methods (.NET 11 Preview 4 runtime notes). Hot tasks, context capture and cancellation tokens — everything this lesson teaches — keep their meaning. • Verified

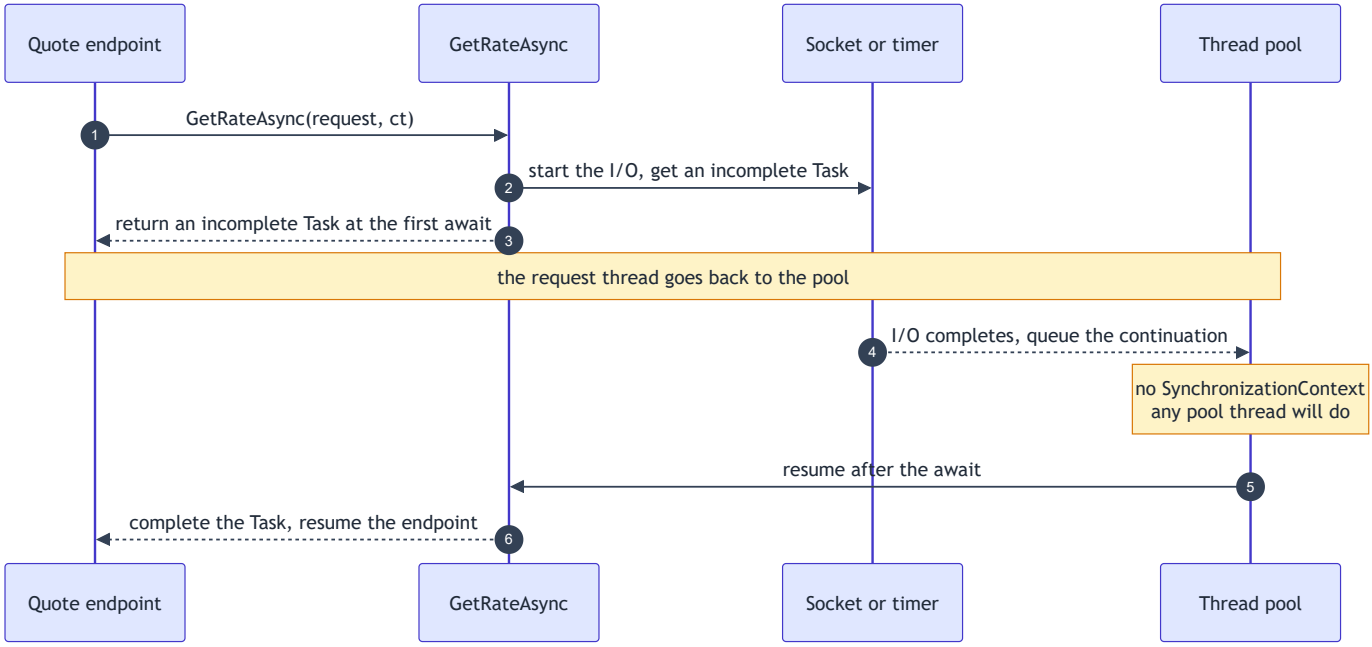
.NET has no virtual threads, by decision. Java 21 made blocking code cheap with virtual threads; the .NET team prototyped the same idea, green threads, and in 2023 put the experiment on hold to keep improving `async/await`, mainly because a second programming model was a larger cost than the gain (the .NET green-threads experiment: results). • Verified The Task model in this lesson is the model you will keep using.

2.1 · Concept map — coroutines, Reactor and the event loop → .NET

You already know	In .NET	Kind	Watch for
Kotlin `suspend fun` · TS `async function`	`async Task<T>` method	• Renamed	`async` is an implementation detail; callers see only the `Task<T>`
`async {}` → `Deferred` · JS `Promise`	a started `Task<T>`	• Same idea	Hot: the work starts when the method is called, not when it is awaited
Reactor `Mono<T>`	`Task<T>`	• Trap	A `Mono` re-runs per subscriber; a `Task` runs once, and awaiting it again returns the same result
`coroutineScope {}` — structured concurrency	no equivalent	• Different	Nothing cancels or awaits children for you: pass a token and `await Task.WhenAll` (§4)
`Job.cancel()` · `AbortController`	`CancellationTokenSource.Cancel()`	• Different	Kotlin scopes pass cancellation down for you; every .NET async method takes the token as a parameter, explicitly. `AbortController` is the near-rename (3)
`withTimeout` / `withTimeoutOrNull`	`new CancellationTokenSource(timeout)` · `WaitAsync`	• Trap	`WaitAsync` stops *waiting*; only a token stops the *work* (3.4)
`awaitAll()` · `Promise.all` · `Mono.zip`	`Task.WhenAll`	• Trap	Those fail fast; `WhenAll` waits for every task even after one fails, and `await` rethrows only the first exception (4.7)
`select {}` · `Promise.race`	`Task.WhenAny` · `Task.WhenEach`	• Renamed	`WhenEach` (.NET 9) streams tasks as they complete
Kotlin `Flow<T>`	`IAsyncEnumerable<T>` + `await foreach`	• Same idea	Cold and pull-based; cancellation arrives through an attributed parameter (§5)
Reactor `Flux<T>` with back-pressure	bounded `Channel<T>`	• Different	No reactive operator library in the box; async LINQ is, since .NET 10
`Dispatchers.IO` / `Default`	the thread pool	• Different	No named dispatchers: I/O needs no thread, and `Task.Run` queues CPU work
`Dispatchers.Main` · Node's loop thread	a `SynchronizationContext`	• Trap	Blocking that thread on `.Result` deadlocks it (7.1)
`runBlocking {}`	`.GetAwaiter().GetResult()`	• Trap	Tolerable once in a console `Main` with no context; a starvation risk anywhere else
`synchronized` · kotlinx `Mutex`	`lock` · `SemaphoreSlim`	• Different	`lock` cannot contain an `await`; `SemaphoreSlim.WaitAsync` can (§6)
`GlobalScope.launch` · a floating Promise	`_ = Task.Run(…)`	• Trap	Its exception surfaces only when the GC finalizes the task, if ever (7.4)
`runTest` + `advanceTimeBy`	`FakeTimeProvider.Advance`	• Different	Kotlin makes every `delay` virtual; only .NET code that takes a `TimeProvider` sees the fake clock (6.6)
`CompletableFuture` · `allOf` · `orTimeout`	`Task` · `Task.WhenAll` · `WaitAsync`	• Trap	Hot like a `Task`; and `orTimeout`, like `WaitAsync`, stops waiting without stopping the work (3.3)

You already know	In .NET	Kind	Watch for
`Mono.block()` · `CompletableFuture.join()`	`.Result` · `.GetAwaiter().GetResult()`	• Trap	Reactor refuses `block()` on its non-blocking threads; .NET compiles it and then hangs a one-thread context or starves the pool (7.1)
`asyncio.TaskGroup` (Python 3.11)	no equivalent	• Different	The group cancels the siblings of a failed task and waits for all; in .NET use a linked token and `WhenAll` (4.7)

2.2 · One await, step by step — a partner call inside ASP.NET Core



solid arrows = calls and resumptions · dashed arrows = a returned Task or a completion signal · no thread is blocked between steps 3 and 5

2.3 · What the compiler builds, written out by hand

C# — samples/L05.AsyncTraps/Traps.cs · #region lowered

```
// async Task<decimal> GetPremiumAsync() { await Task.Delay(50); return 8_933.71m; }
// written out by hand (the real output is a state-machine struct)
public static Task<decimal> GetPremiumLowered()
{
    var result = new TaskCompletionSource<decimal>();
    var awaiter = Task.Delay(50).GetAwaiter();
    if (awaiter.IsCompleted) Resume();    // already done: no suspension at all
    else awaiter.OnCompleted(Resume);     // suspend: this thread is free until the timer fires
    return result.Task;                   // the caller gets a Task straight away

    void Resume()                         // everything after the await
    {
        try { awaiter.GetResult(); result.SetResult(8_933.71m); }   // rethrows a failure
        catch (Exception ex) { result.SetException(ex); }
    }
}
```

An `await` is an awaiter, a completion check and a callback. If the awaited task is already done, execution continues synchronously with no suspension at all. Otherwise `OnCompleted` registers everything after the `await` and the method returns its `Task` at once. `GetResult()` rethrows a failure, which is why `try/catch` works around an `await`. The real compiler output is a state-machine struct driven by `AsyncTaskMethodBuilder`, and it also flows `ExecutionContext` (for example `AsyncLocal` values); `L05.AsyncTraps` runs this version and prints `premium 8,933.71 THB` — the total the MotorQuote running example prices for a Class 1 policy, 550,000 THB insured, driver aged 23 with four claim-free years (an illustrative tariff, not a real one), standing in for a slow rating call.

2.4 · The async toolbox — six types in every .NET service

API view

1 Task and Task<T> C# runtime • Same idea

The unit of async work: a promise for an operation that is **already running**.

KOTLIN	`Deferred<T>` from `async {}`
TS	`Promise<T>`
ENDS AS	`RanToCompletion` · `Faulted` · `Canceled`
RULE	Return it and await it; never block on it with `.Result`

2 ValueTask<T> C# runtime • Different

A struct result that avoids a heap allocation when the answer is often ready at once.

USE WHEN	A profiled hot path completes synchronously most of the time
RULE	Await it once, never twice and never concurrently
SEEN IN	`IAsyncEnumerator.MoveNextAsync`, `ChannelReader.WaitToReadAsync`
DEFAULT	Return `Task<T>` unless a profile says otherwise

3 CancellationToken C# architecture • Different

The cooperative stop signal you pass as the last parameter of every async call.

KOTLIN	`Job.cancel()` — but no scope passes it for you
OWNER	`CancellationTokenSource`: `Cancel`, `CancelAfter`, linked sources
SIGNALS	`OperationCanceledException` from whoever observes it
TRAP	A method without the parameter cannot be stopped

2.4 · The async toolbox (continued)

API view

4 IAsyncEnumerable<T> C# Java/Kotlin • Same idea

A cold, pull-based async sequence, consumed with `await foreach`.

KOTLIN	`Flow<T>`
LINQ	In the box since .NET 10 (`System.Linq.AsyncEnumerable`)
CANCEL	A parameter marked `[EnumeratorCancellation]`
VB	No `Await For Each`: drive the enumerator by hand (7.6)

5 Channel<T> C# architecture • Same idea

An async producer/consumer queue, bounded or unbounded.

KOTLIN	`Channel(capacity)`
WHEN FULL	`Wait` (default) · `DropOldest` · `DropNewest` · `DropWrite`
IN-BOX	Part of the shared framework since .NET Core 3.0
ENDS	`Writer.Complete()` lets readers drain, then stop

6 TimeProvider C# runtime • Different

An injectable clock: timestamps, timers and delays that a test can move by hand.

KOTLIN	`runTest` virtual time with `advanceTimeBy`
SINCE	.NET 8, in the framework
ACCEPTED	`Task.Delay`, `WaitAsync`, `CancellationTokenSource`
TESTS	`FakeTimeProvider.Advance()` fires due timers

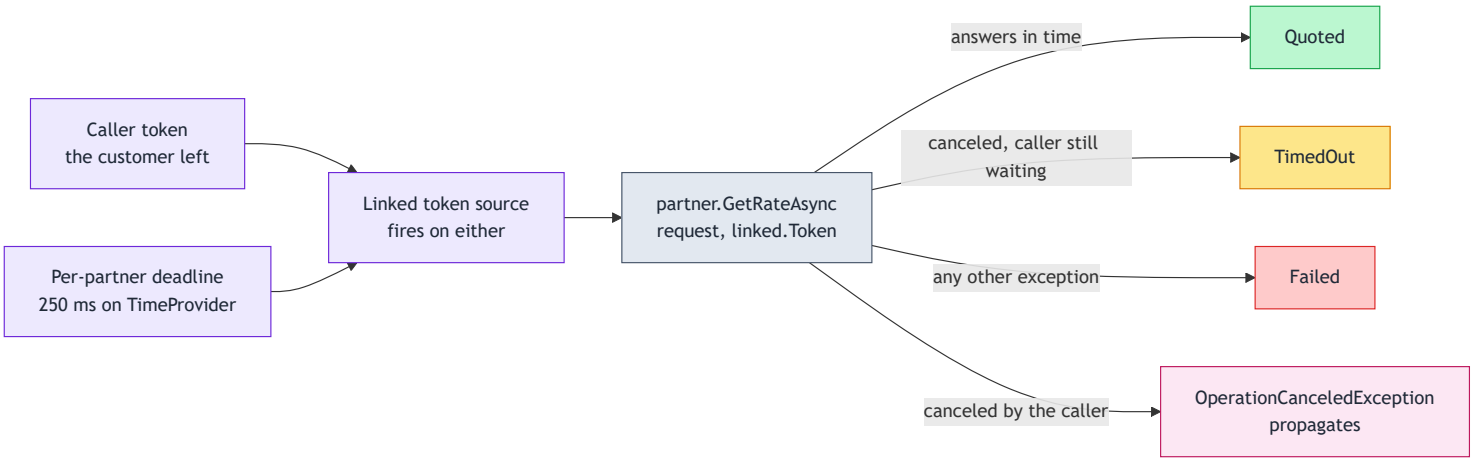
3 · Cancellation and timeouts — tokens, not scopes

Cancellation in .NET is a token passed down by hand, and nothing stops unless the code holding the token checks it. A `CancellationTokenSource` owns the signal; its `Token` travels as the last parameter of every async method down to `HttpClient`, EF Core or `Task.Delay`, which observe it and throw `OperationCanceledException` (cancellation in managed threads). Kotlin propagates cancellation through the coroutine scope; in .NET a method that forgets the parameter is a method nobody can stop.

A timeout is a token that cancels itself, so each partner call needs two tokens. The caller's token says the customer has gone; a per-partner deadline says this partner is too slow. `CreateLinkedTokenSource` combines them, and an exception filter tells them apart afterwards: if the caller's token is not cancelled, the deadline fired, and the partner is recorded as `TimedOut` instead of the whole quote being abandoned.

`WaitAsync` is the `withTimeout` look-alike that cancels nothing. It returns a task that fails with `TimeoutException` when time is up, but the operation underneath keeps running and its result is discarded (Task.WaitAsync, .NET 6; the `TimeProvider` overloads arrived in .NET 8). Use it only for work that takes no token, and budget for that work finishing anyway.

3.1 · Two tokens, four outcomes — one partner call



violet = tokens · slate = the partner call · green / amber / red = recorded outcomes · rose = the only case that escapes to the caller

3.2 · A per-partner deadline on an injected clock

C# samples/L05.PartnerRates/RateFanOut.cs · #region per-partner-timeout

```
private async Task<PartnerOutcome> QueryOneAsync(
    IPartnerRateProvider partner, QuoteRequest request, CancellationToken ct)
{
    long started = time.GetTimestamp();
    // this partner's own deadline, on the injected clock (tests move it by hand)
    using var deadline = new CancellationTokenSource(perPartnerTimeout, time);
    // cancelled when EITHER the caller gives up OR this partner's deadline passes
    using var linked = CancellationTokenSource.CreateLinkedTokenSource(ct, deadline.Token);
    try
    {
        var premium = await partner.GetRateAsync(request, linked.Token).ConfigureAwait(false);
        return new Quoted(partner.PartnerId, time.GetElapsedTime(started), premium);
    }
    catch (OperationCanceledException) when (!ct.IsCancellationRequested)
    {
        // our deadline fired, not the caller's token: a partner problem, not the caller's
        return new TimedOut(partner.PartnerId, time.GetElapsedTime(started));
    }
    catch (Exception ex) when (ex is not OperationCanceledException)
    {
        return new Failed(partner.PartnerId, time.GetElapsedTime(started), ex.Message);
    }
}
```

Three details carry the design. The deadline is a `CancellationTokenSource` built on the injected `TimeProvider`, so tests can fire it without waiting. Both token sources are `using` declarations: a linked source registers callbacks on its parents and must be disposed. And the two `when` filters decide who asked to stop — a cancellation the caller did not request is this partner's timeout, and a caller cancellation passes straight through both filters and out of the method.

3.3 · Four ways to give up — which ones stop the work

API	Stops the partner call?	What the caller sees	Use it for
`new CancellationTokenSource(timeout, time)`	Yes, if the call observes the token	`OperationCanceledException`	A deadline per call, testable with a fake clock

API	Stops the partner call?	What the caller sees	Use it for
`CreateLinkedTokenSource(ct, deadline.Token)`	Yes — whichever token fires first	`OperationCanceledException`	Combining the caller's token with your own deadline
`cts.CancelAfter(timeout)`	Yes, if the call observes the token	`OperationCanceledException`	Adding a deadline to a source you already hold
`task.WaitAsync(timeout, time)`	**No** — only the waiting stops	`TimeoutException`	Work that accepts no token; the work runs on unobserved

3.4 · Proof on a fake clock — WaitAsync gives up, the partner does not

C# samples/L05.PartnerRates.Tests/FanOutTests.cs · #region wait-async

```
[Fact]
public async Task WaitAsync_stops_waiting_but_does_not_stop_the_call()
{
    var time = new FakeTimeProvider();
    var stubborn = new SimulatedPartner("P99", TimeSpan.FromSeconds(2), 200m,
        PartnerBehaviour.IgnoresCancellation, time);

    var call = stubborn.GetRateAsync(Request, CancellationToken.None);
    var waiting = call.WaitAsync(Timeout, time);
    time.Advance(Timeout);

    await Assert.ThrowsAsync<TimeoutException>(() => waiting);
    Assert.False(call.IsCompleted);             // we gave up; the partner did not

    time.Advance(TimeSpan.FromSeconds(2));
    Assert.Equal(13_000m, (await call).Amount); // it finished anyway, unobserved
}
```

The test moves time instead of sleeping. `Advance(Timeout)` fires the `WaitAsync` timer, so awaiting `waiting` throws `TimeoutException` — yet `call.IsCompleted` is still false. Two more virtual seconds later the stubborn partner returns its 13,000.00 THB quote to nobody. That orphaned call is the cost of a timeout without a token: under load it is a connection, a thread-pool work item and a partner's rate limit spent on an answer you threw away.

4 · Fan-out — 30 partners, one deadline

Start every call first, then await them together: the fan-out lasts as long as the slowest partner you are prepared to wait for, not the sum of all of them. `partners.Select(p => QueryOneAsync(…))` only describes the calls; the collection expression `[.. …]` around it runs them, and each call returns a task that is already running. `Task.WhenAll` then waits for all 30. The captured run finished in 263 ms, where awaiting the partners one after another would have waited 4,982 ms. The classic mistake is an `await` inside a `foreach` — sequential code that only looks asynchronous.

There is no `coroutineScope` to clean up after you. In Kotlin a failing child cancels its siblings and the scope waits for all of them. `Task.WhenAll` waits for every task but cancels none, and `await` rethrows only the first exception while the rest stay on the task's `Exception` (Task.WhenAll). That is why `QueryOneAsync` never throws for a partner problem: every call ends as a `Quoted`, `TimedOut` or `Failed` record, and one bad partner cannot sink the quote.

Three shapes cover a real rating desk. *Everything by a deadline* is `WhenAll` over per-call timeouts. *Results as they arrive* is `Task.WhenEach` (.NET 9), an `IAsyncEnumerable` that yields each task the moment it completes and replaces the `WhenAny`-in-a-loop pattern that re-scans the list after every completion (Task.WhenEach). *First good-enough answer* reads that stream and cancels the rest. When a partner contract caps concurrent calls, `Parallel.ForEachAsync` with `MaxDegreeOfParallelism` keeps that many in flight (Parallel.ForEachAsync).

4.1 · The same fan-out — Kotlin coroutines vs C#

Kotlin — for comparison the coroutine fan-out you already write

```kotlin
suspend fun queryAll(request: QuoteRequest)
        : List<PartnerOutcome> = coroutineScope {
    // async {} starts all 30 children at once
    partners.map { p -> async { queryOne(p, request) } }
        .awaitAll()
}   // the scope cannot end while a child is running

fun best(all: List<PartnerOutcome>): Quoted? =
    all.filterIsInstance<Quoted>()
        .minByOrNull { it.premium.amount }
```

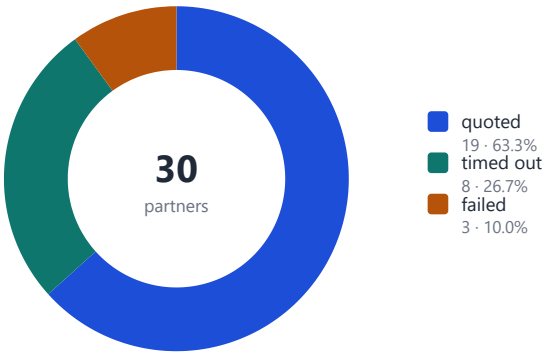
C# samples/L05.PartnerRates/RateFanOut.cs · #region fan-out

```csharp
public async Task<IReadOnlyList<PartnerOutcome>>
    QueryAllAsync(QuoteRequest request,
                  CancellationToken ct = default)
{
    // start all 30 calls first: no await inside the Select
    Task<PartnerOutcome>[] calls = [.. partners
        .Select(p => QueryOneAsync(p, request, ct))];
    // then wait for every one of them, together
    return await Task.WhenAll(calls).ConfigureAwait(false);
}

public static Quoted? Best(IEnumerable<PartnerOutcome> all) =>
    all.OfType<Quoted>().MinBy(q => q.Premium.Amount);
```

Line for line the same shape; the guarantees differ. `coroutineScope` guarantees no child outlives it and cancels the siblings of a failed child. The C# version has no scope: the 30 tasks live on their own, and `WhenAll` only waits. `Select` is lazy; materialising it into an array is what starts the tasks, which is why the code keeps an array rather than a re-enumerable `IEnumerable` (enumerate that twice and every partner is called twice). The C# code stays safe because `QueryOneAsync` (3.2) turns every partner failure into a value and bounds every call with a deadline — the discipline the Kotlin scope gave you for free. `ConfigureAwait(false)` marks library code (7.1). Kotlin coroutine basics.

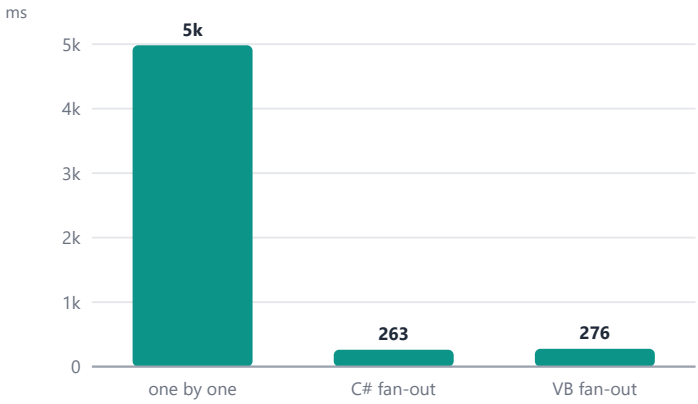
4.2 · What 30 partners returned



one fan-out with a 250 ms deadline per partner · captured from L05.RateDesk and asserted by FanOutTests

19 quotes arrived in time; 8 partners were cut off and 3 failed — and the quote still succeeded. The timed-out slice is the six slow partners plus the two that never answer. • Measured — the counts are identical in the C# and VB runs and in the test that runs on a fake clock.

4.3 · Waiting for 30 partners



one by one = the sum of each partner's wait, capped at the deadline · fan-out = measured wall clock

About 19× faster, and bounded by the deadline rather than by the partner count. The fan-out bars sit just above 250 ms: the deadline plus start-up and scheduling. Add a 31st partner and they stay put; the one-by-one bar grows. • Measured on the build machine — wall-clock figures vary from run to run.

4.4 · Results as they arrive — Task.WhenEach

C# samples/L05.PartnerRates/RateFanOut.cs · #region when-each

```csharp
public async IAsyncEnumerable<PartnerOutcome> StreamAsync(
    QuoteRequest request, [EnumeratorCancellation] CancellationToken ct = default)
{
    Task<PartnerOutcome>[] calls = [.. partners.Select(p => QueryOneAsync(p, request, ct))];
    // .NET 9: hands back each task as it completes — no WhenAny loop re-scanning the list
    var inCompletionOrder = Task.WhenEach(calls).WithCancellation(ct).ConfigureAwait(false);
    await foreach (Task<PartnerOutcome> done in inCompletionOrder)
        yield return await done.ConfigureAwait(false);   // already complete: no wait
}
```

`WhenEach` turns a batch of tasks into an async stream that yields each one as it completes. Each done task has already finished, so `await done` returns at once and unwraps its result. `[EnumeratorCancellation]` lets a consumer pass a token with `WithCancellation`; without the attribute the compiler warns (CS8425), which this repository treats as an error. The test `WhenEach_hands_back_each_outcome_the_moment_it_completes` advances a fake clock partner by partner and asserts the order — which the API does not promise for tasks that complete at the same instant; the test can assert it only because the fake clock finishes the partners one at a time.

4.5 · First good-enough quote, then cancel the rest

C# samples/L05.PartnerRates/RateFanOut.cs · #region first-good-enough

```
public async Task<Quoted?> FirstAtOrBelowAsync(
    QuoteRequest request, decimal targetPremium, CancellationToken ct = default)
{
    using var stop = CancellationTokenSource.CreateLinkedTokenSource(ct);
    await foreach (var outcome in StreamAsync(request, stop.Token).ConfigureAwait(false))
    {
        if (outcome is Quoted q && q.Premium.Amount <= targetPremium)
        {
            await stop.CancelAsync().ConfigureAwait(false);   // tell every other partner to stop
            return q;
        }
    }
    return null;
}
```

Stop paying for answers you no longer need. The loop reads outcomes as they arrive and, at the first premium at or below the target, cancels the linked source that every remaining partner call is listening to. `CancelAsync` (.NET 8) runs the registered callbacks without blocking this thread. The matching test counts the calls that observed cancellation and requires it to equal the number of partners still working at that virtual instant.

4.6 · When a partner contract caps concurrent calls

C# samples/L05.PartnerRates/RateFanOut.cs · #region throttle

```
public async Task<IReadOnlyList<PartnerOutcome>> QueryAllThrottledAsync(
    QuoteRequest request, int maxConcurrent, CancellationToken ct = default)
{
    var results = new ConcurrentBag<PartnerOutcome>();
    var options = new ParallelOptions
    {
        MaxDegreeOfParallelism = maxConcurrent,   // e.g. a shared outbound connection budget
        CancellationToken = ct,
    };
    // at most maxConcurrent partner calls in flight at any moment
    await Parallel.ForEachAsync(partners, options, async (partner, token) =>
        results.Add(await QueryOneAsync(partner, request, token).ConfigureAwait(false)))
        .ConfigureAwait(false);
    return [.. results.OrderBy(r => r.PartnerId)];
}
```

`Parallel.ForEachAsync` keeps at most `maxConcurrent` calls in flight, where `WhenAll` starts all 30. It is the `flatMapMerge(concurrency)` of this lesson. The body runs on several threads at once, so results go into a `ConcurrentBag` and are sorted afterwards. Each partner's deadline starts when its own call starts, not when it queues, so a throttled desk can take longer than one deadline. The test `Throttled_fan_out_never_exceeds_the_concurrency_budget` wraps every partner in a probe and requires the peak number of calls in flight to stay within the budget.

4.7 · What Task.WhenAll does — and what Kotlin engineers expect it to do

Task.WhenAll does

- Wait for every task, including the ones still running after another has failed
- Return all results in the order the tasks were passed in
- Fault with every task's exception collected on `Task.Exception`
- Hold no thread while it waits

Task.WhenAll does not

- Cancel the other tasks when one fails — pass a token and cancel it yourself
- Rethrow more than the first exception from `await`
- Start anything — the array it receives was already running (hand it a lazy `IEnumerable` and enumeration, not `WhenAll`, starts the calls)
- Limit concurrency — 30 partners means 30 calls in flight

5 · Streams and pipelines — IAsyncEnumerable and Channels

`IAsyncEnumerable<T>` is Kotlin's `Flow`: cold, pull-based and consumed with `await foreach`. An `async` method that uses `yield return` produces one; nothing runs until something iterates it, and each `MoveNextAsync` pulls one item. .NET 10 put LINQ for async streams in the box as `System.Linq.AsyncEnumerable`, replacing the community `System.Linq.Async` package — remove that reference when you upgrade, or calls such as `Take` become ambiguous (System.Linq.AsyncEnumerable in .NET 10).

• Verified

Cancellation has to be threaded in explicitly. Kotlin's `flow {}` checks for cancellation on every `emit` (kotlinx flow builder). A C# async iterator sees a token only through a parameter marked `[EnumeratorCancellation]`, which receives both the token passed to the method and one passed later with `WithCancellation`.

When producers and consumers run at different speeds, put a bounded `Channel<T>` between them. The notification dispatcher reads quote events, routes each to an SMS, email or LINE lane, and sends with one worker per lane. Every channel is bounded, and with the default `FullMode = Wait` a full channel makes its writer wait — back-pressure that travels upstream until the event reader slows down, instead of memory growing until the process fails. `DropOldest`, `DropNewest` and `DropWrite` trade that safety for lost messages (Channels). It is the in-process version of the SQS/SNS fan-in you run between services; lesson 08 (ASP.NET Core Web APIs) hosts it as a `BackgroundService`.

5.1 · An event stream — Kotlin Flow vs IAsyncEnumerable

Kotlin — for comparison the Flow you already write

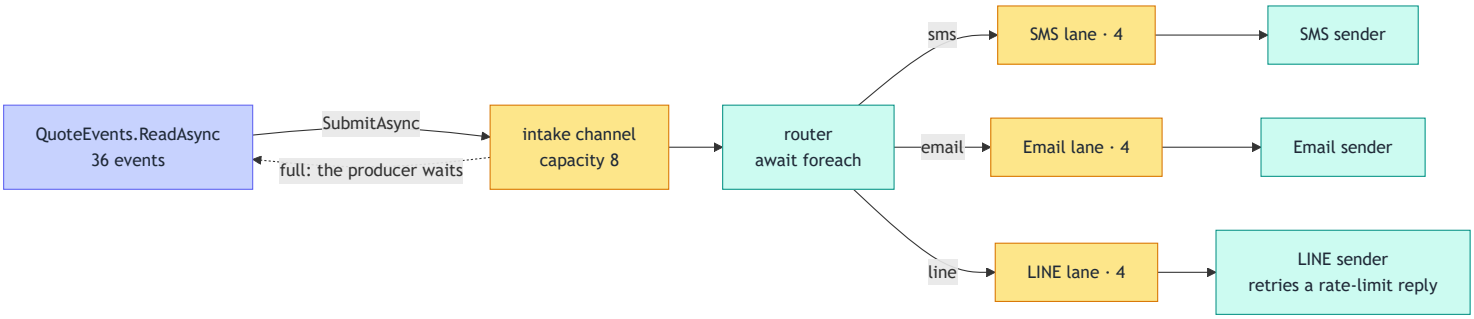
```kotlin
// cold, like IAsyncEnumerable: nothing runs
// until someone collects it
fun readQuoteEvents(count: Int): Flow<Notification> =
    flow {
        for (id in 1..count) {
            yield()          // stand-in for an event read
            val channel = when (id % 4) {
                0 -> NotifyChannel.EMAIL
                2 -> NotifyChannel.SMS
                else -> NotifyChannel.LINE
            }
            val quoteId = "Q-${1000 + id}"
            // emit() also checks for cancellation
            emit(Notification(id, quoteId, channel,
                "Quote $quoteId is ready"))
        }
    }
```

C# samples/L05.Notifications/Notification.cs · #region async-stre…

```csharp
// cold and pull-based, like a Kotlin Flow: nothing
// runs until someone iterates it with await foreach
public static async IAsyncEnumerable<Notification> ReadAsync(
    int count,
    [EnumeratorCancellation] CancellationToken ct = default)
{
    for (var id = 1; id <= count; id++)
    {
        await Task.Yield();   // stand-in for an event read
        ct.ThrowIfCancellationRequested();
        var channel = (id % 4) switch
        {
            0 => NotifyChannel.Email,
            2 => NotifyChannel.Sms,
            _ => NotifyChannel.Line,   // LINE is the default
        };
        var quoteId = $"Q-{1000 + id}";
        yield return new Notification(id, quoteId, channel,
            $"Quote {quoteId} is ready");
    }
}
```

Same shape, one extra parameter. `flow {}` + `emit` becomes an `async` method with `yield return`; `collect` becomes `await foreach`. The difference is the token: Kotlin inherits cancellation from the collector's scope, while C# receives it as a parameter and checks it with `ThrowIfCancellationRequested`. `await Task.Yield()` plays the role of Kotlin's `yield()`: it forces the rest of the loop to continue asynchronously.

5.2 · The notification dispatcher — bounded at every stage



indigo = async stream · amber = bounded channels (capacities from Program.cs) · teal = workers · one lane and one sender per channel · dashed = back-pressure: a full intake makes SubmitAsync wait

5.3 · Bounded channels and a producer that waits

C# — samples/L05.Notifications/Dispatcher.cs · #region bounded-channel

```
public Dispatcher(int intakeCapacity, int laneCapacity)
{
    // Wait (the default FullMode): a full intake pauses the producer instead of dropping work
    _intake = Channel.CreateBounded<Notification>(new BoundedChannelOptions(intakeCapacity)
    {
        FullMode = BoundedChannelFullMode.Wait,
        SingleReader = true,                    // only the router reads the intake
    });
    _lanes = Enum.GetValues<NotifyChannel>()
        .ToDictionary(c => c, _ => Channel.CreateBounded<Notification>(laneCapacity));
}

public async ValueTask SubmitAsync(Notification n, CancellationToken ct)
{
    if (_intake.Writer.TryWrite(n)) return;     // room: no waiting at all
    Interlocked.Increment(ref _producerWaits);  // full: count it, then wait for space
    await _intake.Writer.WriteAsync(n, ct);
}

public void Complete() => _intake.Writer.Complete();   // "no more notifications"
```

`TryWrite` first, `WriteAsync` only when full. A successful `TryWrite` costs no await at all; a full intake falls through to `WriteAsync`, which completes when the router makes room — the dispatcher counts those waits. `SingleReader = true` is a promise that lets the channel use a cheaper implementation; `Complete()` is how the producer says it is done.

5.4 · Router and senders — completion flows downstream

C# — samples/L05.Notifications/Dispatcher.cs · #region pipeline

```
public async Task<DispatchReport> RunAsync(CancellationToken ct)
{
    // start one sender per lane; each waits for work
    var senders = _lanes.Select(lane => SendAllAsync(lane.Key, lane.Value.Reader, ct)).ToArray();

    // the router: ends when the intake is completed AND empty
    await foreach (var n in _intake.Reader.ReadAllAsync(ct))
        await _lanes[n.Channel].Writer.WriteAsync(n, ct);   // waits if that lane is full

    foreach (var lane in _lanes.Values) lane.Writer.Complete();  // let the senders drain
    var sent = await Task.WhenAll(senders);
    var perChannel = sent.ToDictionary(s => s.Channel, s => s.Count);
    return new DispatchReport(perChannel, _retries, _producerWaits);
}
```

Shutdown is a relay, not a kill. The router's `await foreach` ends only when the intake is completed *and* empty; it then completes each lane, each sender drains its lane and returns its count, and `Task.WhenAll` collects the three results. Nothing is dropped, and the program exits by itself.

5.5 · What the dispatcher delivered



36 quote events · intake capacity 8 · lane capacity 4 · captured from L05.Notifications

All 36 notifications were delivered, 4 of them after a retry, and the producer had to wait for space about 10 times. Those waits are back-pressure working: the event reader was paused, and no queue grew past its bound. LINE carries half the traffic because the routing rule sends every odd-numbered event there, and a LINE message whose event number is a multiple of 5 is rejected once by the simulated rate limit. The wait count is a timing result: 10 or 11 across repeated runs on the build machine. • **Measured**

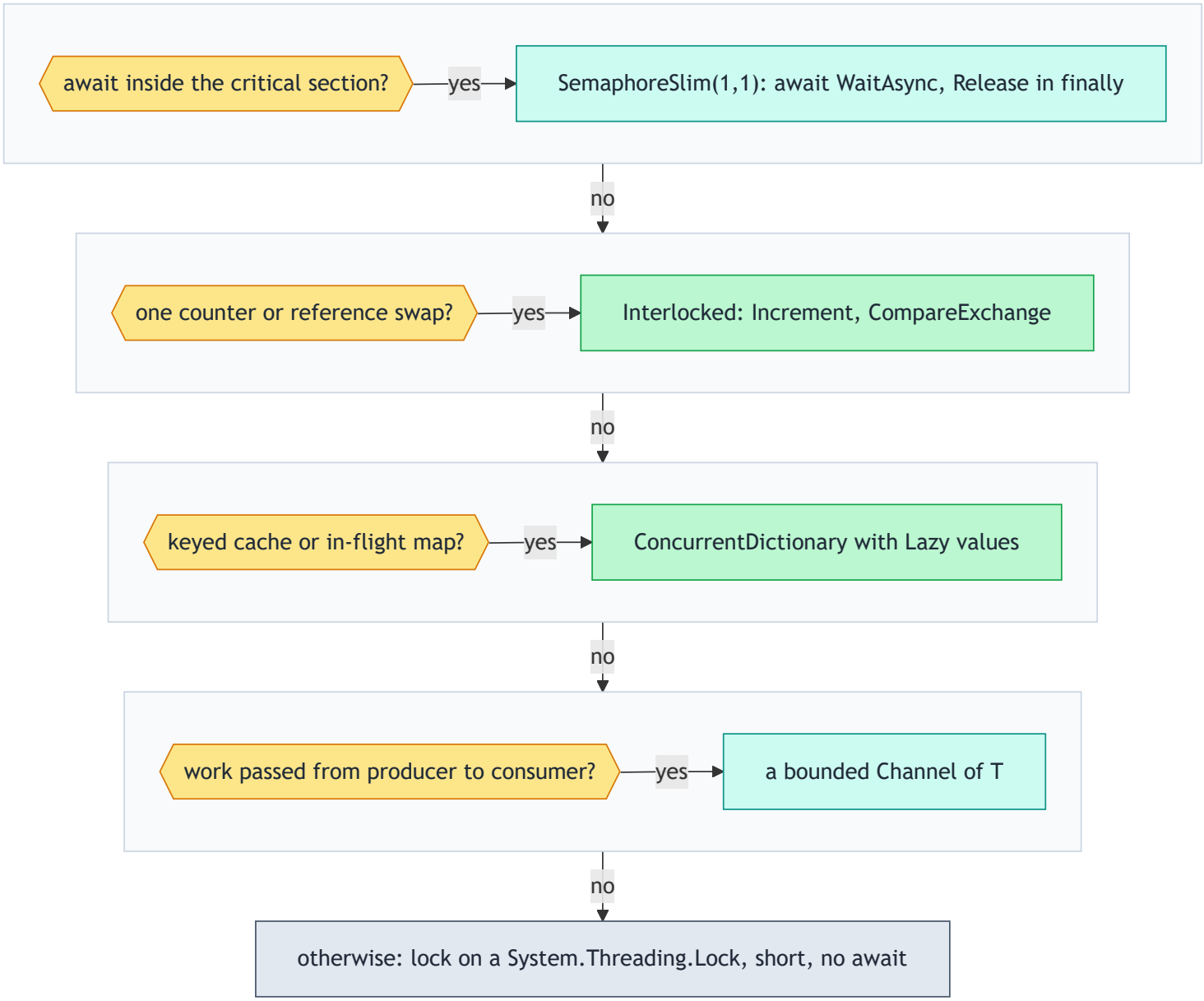
6 · Shared state and testable time

Async code still runs on many threads at once, so shared state needs the discipline you know from Java — plus one new rule: never hold a lock across an `await`. The code after an `await` may resume on a different thread from the one that took the lock, so C# rejects `await` inside a `lock` body (the lock statement) and VB rejects it inside `SyncLock` (7.7). Around an `await`, use `SemaphoreSlim.WaitAsync`, which waits without holding a thread (6.5).

Pick the smallest tool that makes the operation atomic. One counter is `Interlocked.Increment`. A read-compare-write — is this quote cheaper than the best so far? — needs a lock; on .NET 9 and later, lock a dedicated `System.Threading.Lock`, which the C# 13 compiler turns into `EnterScope()` (System.Threading.Lock). A keyed cache or an in-flight map is a `ConcurrentDictionary`, whose `GetOrAdd` may run its factory more than once under a race (ConcurrentDictionary.GetOrAdd) — so wrap a partner call in `Lazy<T>`.

Time is shared state too, and `TimeProvider` makes it injectable. Every timer in `L05.PartnerRates` — each partner's simulated latency and each per-partner deadline — goes through a `TimeProvider`. Production passes `TimeProvider.System`; tests pass `FakeTimeProvider` from `Microsoft.Extensions.TimeProvider.Testing` and call `Advance`, so a 250 ms timeout costs no wall-clock time and cannot flake on a slow build agent (What is TimeProvider).

6.1 · Choosing a primitive for shared state



one row per question, asked top to bottom · amber = question · teal = safe across await · green = lock-free · slate = must not span an await

6.2 · Interlocked for counters, Lock for read-compare-write

C# — samples/L05.PartnerRates/RateStats.cs · #region stats

```
private readonly Lock _gate = new();   // .NET 9 / C# 13: a dedicated lock type
private Quoted? _best;
private int _quoted, _timedOut, _failed;

public void Record(PartnerOutcome outcome)
{
    switch (outcome)
    {
        case Quoted q:
            Interlocked.Increment(ref _quoted);      // one integer: no lock needed
            lock (_gate)                             // read-compare-write: needs the lock
            {
                if (_best is null || q.Premium.Amount < _best.Premium.Amount)
                    _best = q;
            }
            break;
        case TimedOut:
            Interlocked.Increment(ref _timedOut);
            break;
        case Failed:
            Interlocked.Increment(ref _failed);
            break;
    }
}
```

Two tools, chosen per field. Each counter changes with one atomic increment, so `Interlocked` needs no lock. Updating `_best` reads, compares and writes — three steps another thread could interleave — so it happens inside `lock (_gate)`, which compiles to `_gate.EnterScope()` for a `Lock`. The test `RateStats_counts_exactly_under_parallel_writers` records 30,000 outcomes from `Parallel.For` and requires exact counts.

6.3 · One partner call per key, however many callers

C# — samples/L05.PartnerRates/RateStats.cs · #region single-flight

```
private readonly ConcurrentDictionary<TKey, Lazy<Task<TValue>>> _inFlight = new();

public Task<TValue> RunAsync(TKey key, Func<TKey, Task<TValue>> call)
{
    // GetOrAdd can run its factory twice under a race — so the factory only builds a cheap
    // Lazy wrapper, and only the Lazy that won is ever started: one partner call per key
    var lazy = _inFlight.GetOrAdd(key, k => new Lazy<Task<TValue>>(() => call(k)));
    return lazy.Value;
}
```

The `Lazy` wrapper is the whole trick. Under a race `ConcurrentDictionary.GetOrAdd` may invoke the factory on two threads, but only one value is stored; building a cheap `Lazy` is harmless, and only the stored `Lazy` is ever started. The test fires 50 concurrent callers for one vehicle key and asserts exactly one partner call: stampede protection, as Caffeine gives you on the JVM.

6.4 · Concurrency primitives next to their JVM counterparts

Primitive	JVM / Kotlin counterpart	Across an await?	In the samples
`Interlocked`	`AtomicInteger`, `AtomicReference`	Yes — no lock is held	6.2 `RateStats` counters
`lock` on `System.Threading.Lock`	`synchronized`, `ReentrantLock`	**No** — a compile error	6.2 `RateStats` best quote
`SemaphoreSlim`	`Semaphore`, kotlinx `Mutex`	Yes, with `WaitAsync`	6.5 `PartnerToken`
`ConcurrentDictionary`	`ConcurrentHashMap`	Yes	6.3 `SingleFlight`
`ConcurrentBag` · `ConcurrentQueue`	`ConcurrentLinkedQueue`	Yes	4.6 throttled fan-out
`Channel<T>`	`BlockingQueue`, Kotlin `Channel`	Yes	5.3, 5.4 notification dispatcher
`Parallel.ForEachAsync`	`flatMapMerge(concurrency)`	Yes	4.6 `QueryAllThrottledAsync`

6.5 · A lock you can hold across an await — SemaphoreSlim

C# samples/L05.PartnerRates/PartnerToken.cs · #region token-gate

```
public sealed class PartnerToken(
    TimeProvider time, Func<CancellationToken, Task<string>> fetch, TimeSpan lifetime)
{
    private sealed record Issued(string Value, DateTimeOffset Expires);

    private readonly SemaphoreSlim _gate = new(1, 1);   // one refresh at a time
    private volatile Issued? _current;                  // one reference: no half-written token

    public async Task<string> GetAsync(CancellationToken ct = default)
    {
        if (_current is { } fresh && fresh.Expires > time.GetUtcNow())
            return fresh.Value;                         // fast path: no wait, no lock

        await _gate.WaitAsync(ct).ConfigureAwait(false); // waits without holding a thread
        try
        {
            var issued = _current;                      // the caller ahead may have refreshed
            if (issued is null || issued.Expires <= time.GetUtcNow())
            {
                var value = await fetch(ct).ConfigureAwait(false);   // an await inside the gate
                _current = issued = new Issued(value, time.GetUtcNow() + lifetime);
            }
            return issued.Value;
        }
        finally
        {
            _gate.Release();                            // always, or later callers wait forever
        }
    }
}
```

`SemaphoreSlim(1, 1)` is the mutex that may span an `await`: `WaitAsync` queues callers without holding a thread. It is the job kotlinx `Mutex.withLock` does in a coroutine. `Release` belongs in `finally`, or one failed refresh leaves every later caller waiting for ever. The fast path skips the gate; after the gate the code reads the token again, because the caller ahead may already have refreshed it. The test `PartnerToken_is_fetched_once_for_fifty_callers_and_again_after_it_expires` lets fifty callers share one fetch, then moves a fake clock past the expiry and requires exactly one more.

6.6 · A 250 ms deadline tested in microseconds

C# samples/L05.PartnerRates.Tests/FanOutTests.cs · #region fake-time

```
[Fact]
public async Task Fan_out_ends_at_the_timeout_not_the_sum_of_latencies()
{
    var time = new FakeTimeProvider();
    var fanOut = new RateFanOut(PartnerCatalog.Create(time), time, Timeout);

    var call = fanOut.QueryAllAsync(Request);   // all 30 timers registered here
    time.Advance(TimeSpan.FromMilliseconds(249));
    Assert.False(call.IsCompleted);             // slow partners still out

    time.Advance(TimeSpan.FromMilliseconds(1)); // deadlines fire now
    var outcomes = await call;                  // no real time has passed

    Assert.Equal(19, outcomes.OfType<Quoted>().Count());
    Assert.Equal(8, outcomes.OfType<TimedOut>().Count());
    Assert.Equal(3, outcomes.OfType<Failed>().Count());
}
```

249 ms proves nothing has fired; 1 ms more proves everything does. Every partner timer and deadline is registered on the `FakeTimeProvider` when `QueryAllAsync` starts, and fires only inside `Advance`. The three counts are exact because no wall clock is involved — the same 19 / 8 / 3 the real-clock run printed in 4.2. The test project has 11 tests and 0 calls to `Thread.Sleep` or `Task.Delay`. • **Measured**

7 · Traps — and what Visual Basic cannot say

Blocking on a task with `.Result` or `.Wait()` deadlocks wherever a SynchronizationContext owns a single thread. The blocked thread is the only one allowed to run the continuation that would unblock it. ASP.NET Core has no such context, so the same code appears to work there — and instead holds a pool thread per request. Starvation is quiet rather than loud: in Microsoft's walkthrough the pool grew quickly to 2–3 times the core count and then added only 1–2 threads a second, so under

load requests queue and latency climbs instead of anything failing — the same stall as blocking a Netty event-loop thread in WebFlux (ASP.NET Core best practices · debug ThreadPool starvation). • Verified General-purpose libraries therefore put `ConfigureAwait(false)` on every `await` so they never need the caller's context; application code does not (ConfigureAwait FAQ).

`async void` exists for event handlers only. The caller gets nothing to await, so it cannot catch the method's exception; the exception is re-raised on the SynchronizationContext, or on the thread pool when there is none, where it is likely to terminate the process (async return types). In ASP.NET Core the documentation calls it always a bad practice.

A task nobody awaits hides its failure. `_ = Task.Run(…)` compiles cleanly; if it throws, `TaskScheduler.UnobservedTaskException` is raised only when the garbage collector finalizes the task — the demo in 7.4 has to force a collection to see it — and by default the process carries on (TaskScheduler.UnobservedTaskException). Background work belongs in a hosted service, covered in lesson 08 (ASP.NET Core Web APIs). And inside ASP.NET Core, `await Task.Run(…)` around code already running on a pool thread only adds a scheduling hop.

Visual Basic has had `Async` and `Await` since Visual Studio 2012, and stopped there. The VB compiler rejects `Await` in `Catch`, `Finally` and `SyncLock` (Await operator (Visual Basic)), `Async Sub Main`, async iterators, `Await For Each` and `ValueTask`-returning async functions. It also has no `Await Using`, and a plain `Using` refuses a type that implements only `IAsyncDisposable` — the `await using` from lesson 02 (C# Language Essentials) has no VB spelling. Table 7.7 lists the exact errors, captured by compiling each snippet in `L05.VbRules.Tests`. VB can still consume everything the C# library offers — it just does so by hand: `Await c.DisposeAsync()` after the block, never in `Finally` (7.6).

7.1 · Sync-over-async on a one-thread context

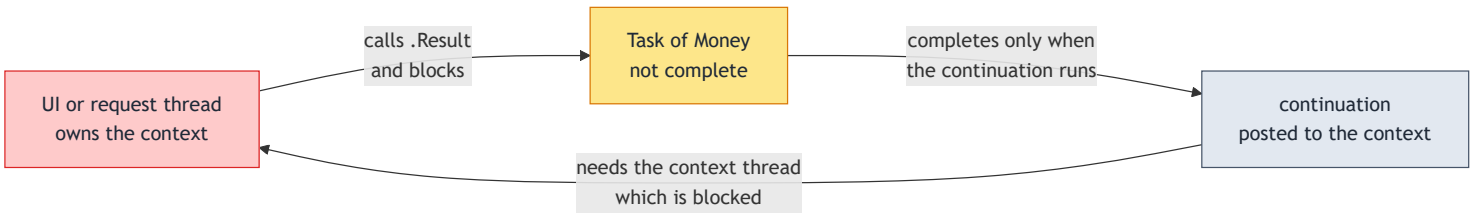
C# — samples/L05.AsyncTraps/Traps.cs · #region deadlock

```
// library code: awaits with or without ConfigureAwait(false)
private static async Task<decimal> GetPremiumAsync(bool captureContext)
{
    await Task.Delay(50).ConfigureAwait(captureContext);
    return 8_933.71m;   // with a captured context, this line must run ON that context's thread
}

// a "UI thread": owns a one-thread context, then blocks on .Result
public static string BlockOnResult(bool captureContext)
{
    var finished = new ManualResetEventSlim();
    var uiThread = new Thread(() =>
    {
        SynchronizationContext.SetSynchronizationContext(new OneThreadContext());
        _ = GetPremiumAsync(captureContext).Result;   // sync-over-async: this thread now waits
        finished.Set();
    }) { IsBackground = true };
    uiThread.Start();
    // the continuation is queued to uiThread, and uiThread is waiting for the continuation
    return finished.Wait(TimeSpan.FromSeconds(1)) ? "completed" : "DEADLOCK (gave up after 1 s)";
}
```

Same method, one argument apart. With `ConfigureAwait(true)` the continuation of `GetPremiumAsync` is posted to the thread's context — the thread blocked in `.Result`, which will never pump it. With `false` it runs on the pool and completes. `OneThreadContext` is a 39-line stand-in for a UI thread or classic ASP.NET; the demo gives up after one second instead of hanging (output in 7.5).

7.2 · Why the deadlock never resolves



a cycle of two waits: the thread waits for the task, the task waits for the thread · red = the blocked thread · amber = the task it waits for · slate = the continuation queued to that thread

7.3 · async void — the exception the caller never sees

C# — samples/L05.AsyncTraps/Traps.cs · #region async-void

```
private static async void SendQuoteEmail()          // async void: nothing to await
{
    await Task.Delay(20);
    throw new InvalidOperationException("SMTP relay refused the message");
}

public static (string CallerSaw, string ContextGot) Run()
{
    var context = new OneThreadContext();
    return context.RunOnOwnThread(() =>
    {
        var callerSaw = "nothing";
        try
        {
            SendQuoteEmail();                       // returns at its first await
        }
        catch (Exception ex)
        {
            callerSaw = ex.GetType().Name;          // never reached
        }
        // the exception is re-thrown on the captured context, not to the caller
        var escaped = context.PumpUntilException(TimeSpan.FromSeconds(2));
        return (callerSaw, escaped?.GetType().Name ?? "nothing");
    });
}
```

The `catch` around the call cannot fire. `SendQuoteEmail` returns at its first `await`; 20 ms later the exception is posted to the captured context, where the demo pumps it out. With no context it would be thrown on the thread pool. Return `Task` and `await` it.

7.4 · A forgotten Task.Run — the failure nobody observes

C# — samples/L05.AsyncTraps/Traps.cs · #region fire-and-forget

```
public static bool Run()
{
    var noticed = false;
    TaskScheduler.UnobservedTaskException += (_, e) => { noticed = true; e.SetObserved(); };

    var forgotten = StartAndForget();                   // nobody awaits the task
    SpinWait.SpinUntil(() => !forgotten.TryGetTarget(out var t) || t.IsCompleted);

    GC.Collect();                   // the failure is reported only when the GC finalizes
    GC.WaitForPendingFinalizers();  // the faulted Task — seconds, hours or never later
    return noticed;
}

[MethodImpl(MethodImplOptions.NoInlining)]
private static WeakReference<Task> StartAndForget() => new(Task.Run(CallSlowPartner));

private static void CallSlowPartner() => throw new TimeoutException("P11 did not answer");
```

The exception is reported only when the garbage collector finalizes the faulted `Task`. `StartAndForget` keeps just a `WeakReference`, and `NoInlining` keeps a strong reference out of the caller's frame, so nothing holds the task. The demo waits until the task has faulted, then forces `GC.Collect()` and `WaitForPendingFinalizers()` so `UnobservedTaskException` fires now, and `SetObserved()` marks it handled. Production code has no such push: the event comes whenever a collection happens to run, or never.

7.5 · The four traps, reproduced

Output — L05.AsyncTraps captured on the build machine

```
await, lowered by hand
  premium 8,933.71 THB
sync-over-async (.Result) on a one-thread context
  await captures the context -> DEADLOCK (gave up after 1 s)
  ConfigureAwait(false)      -> completed
async void throws after its first await
  the caller's catch saw     -> nothing
  the context received       -> InvalidOperationException
fire-and-forget Task.Run that throws
  reported only after a GC   -> True
```

Read each pair as cause and effect. The deadlock line takes one second because the demo waits that long before giving up. `nothing` / `InvalidOperationException` is the async-void exception escaping its caller (7.3). The last line is `True` only because the sample forces a garbage collection (7.4) — in production the unobserved exception appears whenever the GC happens to run, or never.

7.6 · Consuming an async stream — C# vs Visual Basic

C# samples/L05.RateDesk/Program.cs · #region stream

```csharp
// the first 5 answers, as they arrive (Take: .NET 10 ships
// LINQ for IAsyncEnumerable, no System.Linq.Async package)
using var enough = new CancellationTokenSource();
var answers = fanOut.StreamAsync(request, enough.Token);
await foreach (var outcome in answers.Take(5))
{
    Console.WriteLine("    " + Describe(outcome));
}
// leaving the loop does NOT stop the calls it started
await enough.CancelAsync();
```

Visual Basic samples/L05.VbAsync/Program.vb · #region stream

```vb
' no Await For Each in VB: drive the enumerator by hand
Using enough As New CancellationTokenSource()
    Dim e = fanOut.StreamAsync(request, enough.Token).
        GetAsyncEnumerator()
    Dim shown = 0
    While shown < 5 AndAlso Await e.MoveNextAsync()
        Console.WriteLine("    " & Describe(e.Current))
        shown += 1
    End While
    ' no Await inside Finally: dispose explicitly
    Await e.DisposeAsync()
    Await enough.CancelAsync()
End Using
```

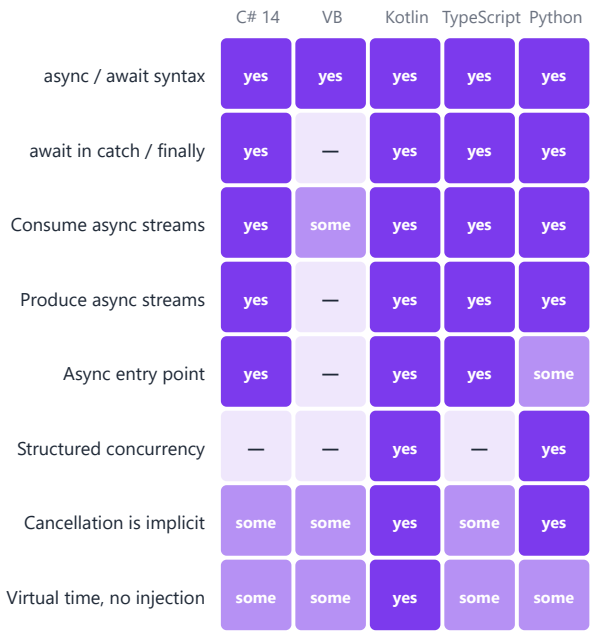
7 lines in C#, 11 in VB, and most of the difference is enumerator plumbing that `await foreach` writes for C#. With no `Await For Each`, VB calls `GetAsyncEnumerator`, loops on `Await e.MoveNextAsync()` and disposes the enumerator itself — after the loop, because an `Await` in a `Finally` block does not compile, so an exception inside the loop would skip the dispose. The C# side uses `Take(5)` from the .NET 10 in-box async LINQ. On both sides, leaving the loop does not stop the partner calls; cancelling the token does.

7.7 · What the Visual Basic compiler rejects

VB construct	Error	Message (captured)	C#
`Await` inside `Catch`	BC36943	'Await' cannot be used inside a 'Catch' statement, a 'Finally' statement, or a 'SyncLock' statement.	Allowed since C# 6
`Await` inside `Finally`	BC36943	(same message)	Allowed since C# 6
`Await` inside `SyncLock`	BC36943	(same message)	Also rejected: no `await` in a `lock` body
`Async Sub Main()`	BC36934	The 'Main' method cannot be marked 'Async'.	`async Task Main` since C# 7.1
`Async Iterator Function`	BC36936	'Async' and 'Iterator' modifiers cannot be used together.	Async streams since C# 8
`Await For Each q In quotes`	BC30201	Expression expected.	`await foreach` since C# 8
`Await Using c As New …`	BC30201	Expression expected.	`await using` since C# 8
`Using c As New …` on a type with only `IAsyncDisposable`	BC36010	'Using' operand of type 'PartnerConnection' must implement 'System.IDisposable'.	`await using` disposes it
`Async Function … As ValueTask(Of T)`	BC36945	The 'Async' modifier can only be used on Subs, or on Functions that return Task or Task(Of T).	`async ValueTask<T>` compiles

VB construct	Error	Message (captured)	C#
`SyncLock` on a `System.Threading.Lock`	`BC37329`	A value of type 'System.Threading.Lock' is not supported in SyncLock. Consider manually calling 'Enter' and 'Exit' methods in a Try/Finally block instead.	`lock` calls `EnterScope()` (C# 13)

7.8 · Async features across the languages you use



yes = the language or its runtime does it for you · some = you pass a token, inject a clock or add a library · — = none · a rubric, not a benchmark

C# hands you the tools, not the automation: structured concurrency, implicit cancellation and ambient virtual time are the rows where Kotlin stays ahead, and VB loses every row that needed syntax after 2012. One rule scores every column: *some* means a token you pass, a clock you inject or a library you add. The VB column's gaps are **• Measured** by compiling (table 7.7); every other cell is an **• Estimate**, the author's judgement. Structured concurrency: Kotlin (`coroutineScope`) and Python (asyncio.TaskGroup, 3.11). C# dates: The history of C#.

8 · Hands-on — run the rating desk in both languages

7 projects, one command. `verify_samples.py` builds every lesson-05 project, runs the 22 tests (the fan-out on a fake clock, the VB compiler rules) and runs the four console apps, each of which exits by itself. Run the two rating desks yourself to watch the fan-out on the real clock.

Two lines will differ on your machine, and that is part of the lesson: the wall clock, and the order of answers that arrive a few milliseconds apart, change from run to run because real timers are coarse. That is why the tests use `FakeTimeProvider`: on a fake clock the order is exact.

8.1 · The same desk — C# vs Visual Basic, one C# library

C# — samples/L05.RateDesk/Program.cs · #region program

```
var fanOut = new RateFanOut(partners, TimeProvider.System,
                            timeout);
var clock = Stopwatch.StartNew();
var outcomes = await fanOut.QueryAllAsync(request);
clock.Stop();

var stats = new RateStats();
foreach (var outcome in outcomes) stats.Record(outcome);

Console.WriteLine($"  quoted {stats.QuotedCount}"
    + $", timed out {stats.TimedOutCount}"
    + $", failed {stats.FailedCount}");
Console.WriteLine($"  best   {stats.Best?.PartnerId}"
    + $" {stats.Best?.Premium}");
```

Visual Basic — samples/L05.VbAsync/Program.vb · #region program

```
Dim fanOut As New RateFanOut(partners, TimeProvider.System,
                             timeout)
Dim clock = Stopwatch.StartNew()
Dim outcomes = Await fanOut.QueryAllAsync(request)
clock.Stop()

Dim stats As New RateStats()
For Each outcome In outcomes
    stats.Record(outcome)
Next

Console.WriteLine($"  quoted {stats.QuotedCount}" &
    $", timed out {stats.TimedOutCount}" &
    $", failed {stats.FailedCount}")
Console.WriteLine($"  best   {stats.Best?.PartnerId}" &
    $" {stats.Best?.Premium}")
```

12 lines in C#, 14 in VB — VB awaits a C# `Task` exactly as C# does. `Dim … As New`, `For Each … Next` and `&` for string concatenation are syntax, not semantics. VB cannot declare `Async Sub Main`, so `L05.VbAsync` blocks once in `Main` with `GetAwaiter().GetResult()` — safe only because a console app has no SynchronizationContext.

8.2 · Commands

Terminal

```
# build, test and run every lesson-05 sample
python 0-script/verify_samples.py --only 5

# the rating desk on the real clock, in C# and in VB
dotnet run --project lesson-05-async-concurrency/samples/L05.RateDesk
dotnet run --project lesson-05-async-concurrency/samples/L05.VbAsync

# the pipeline, the traps, and the tests on virtual time
dotnet run --project lesson-05-async-concurrency/samples/L05.Notifications
dotnet run --project lesson-05-async-concurrency/samples/L05.AsyncTraps
dotnet test lesson-05-async-concurrency/samples/L05.PartnerRates.Tests
```

> **No restore or build step:** `dotnet run` and `dotnet test` do both first; `L05.VbRules.Tests` also downloads the Roslyn VB compiler on first restore.

8.3 · What you should see

Output — C# desk captured on the build machine

```
C#  30 partners, timeout 250 ms
  quoted 19, timed out 8, failed 3
  best   P12 12,090.00 THB
  wall   263 ms concurrently, 4,982 ms one after another
  pool   9 thread-pool threads
  first answers, in completion order:
    P18 quoted 15,210.00 THB
    P04 failed (P04: 503)
    P08 quoted 15,860.00 THB
    P29 quoted 12,155.00 THB
    P22 quoted 17,290.00 THB
  cancelled the partners still working
```

Output — VB desk captured on the build machine

```
VB  30 partners, timeout 250 ms
  quoted 19, timed out 8, failed 3
  best   P12 12,090.00 THB
  wall   276 ms concurrently, 4,982 ms one after another
  pool   5 thread-pool threads
  first answers, in completion order:
    P18 quoted 15,210.00 THB
    P04 failed (P04: 503)
    P08 quoted 15,860.00 THB
    P01 quoted 15,145.00 THB
    P22 quoted 17,290.00 THB
  cancelled the partners still working
```

> **The counts and the best premium must match yours; the timings will not.** The wall clock, the pool size and the order after `P18` and `P04` vary: partners answering 1–3 ms apart can arrive in any order, and `P04`, a failure, still counts as an answer. `pool` is `ThreadPool.ThreadCount` after the run: 9 threads (C#) and 5 (VB) served 30 concurrent calls because no call held a thread while it waited; one blocked thread per partner would need about 30 (7.1).

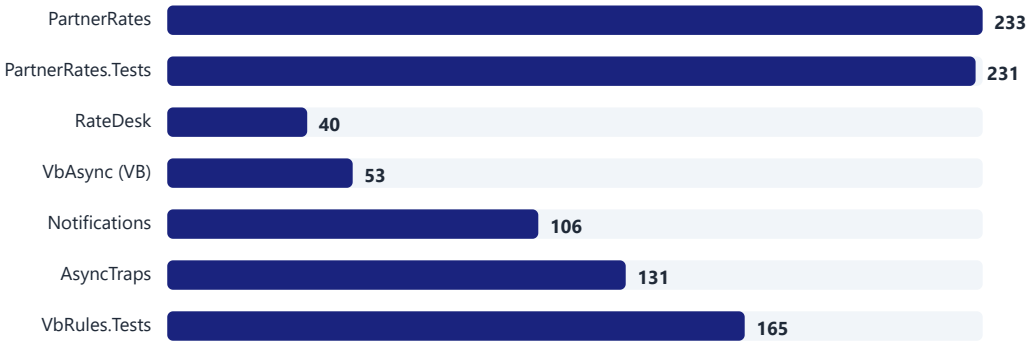
8.4 · What the dispatcher prints

Output — L05.Notifications captured on the build machine

```
36 notifications, intake capacity 8, lane capacity 4
  Sms     9 sent
  Email   9 sent
  Line   18 sent, 4 retried
  producer waited for space 10 time(s)
```

> **The delivery counts are deterministic; the wait count is not.** 36 events split 9 / 9 / 18 by the routing rule and the 4 retries follow from the simulated rate limit. The 10 producer waits depend on capacities and simulated latencies: 10 or 11 on the build machine, a timing result.

8.5 · Lines of code in the lesson-05 samples



non-blank, non-comment lines per project, including test snippets · measured when this PDF was built

Tests, compiler-rule snippets and trap demos make up 527 of the 959 lines, against 233 lines of library. 11 fan-out and shared-state tests and 11 compiler-rule tests back the claims; the two desks (40 and 53 lines) are thin callers of the same library (8.1). • Measured — a size signal, not a quality score.

⚠ Traps

1. **"An `await` inside `foreach` calls the partners concurrently."** It calls them one after another. Start every task first, then `await Task.WhenAll` (4.1).
2. **"`Task.WhenAll` cancels the rest when one fails," like `coroutineScope`.** It cancels nothing and `await` rethrows only the first exception. Turn partner failures into values and cancel through a token (4.7).
3. **"`WaitAsync(timeout)` is `withTimeout`."** It stops the waiting, not the work (test 3.4).
4. **"`.Result` is fine — it works in my API."** ASP.NET Core has no SynchronizationContext, so it does not deadlock there; it starves the thread pool under load, and the same library deadlocks a WinForms caller (7.1).
5. **"An `async void` method is like a TypeScript `async` handler."** Its exception bypasses the caller and can end the process, and a discarded `Task.Run` loses its failure the same way. Return `Task` and await it (7.3, 7.4).

✓ Checkpoint — you are ready for lesson 06 when

1. `python 0-script/verify_samples.py --only 5` reports 7/7 passed on your machine.
2. Using diagram 2.2, you can say which thread resumes after an `await` in ASP.NET Core, a console app and WinForms.
3. You can turn a `foreach` with an `await` inside into a fan-out with per-call deadlines, and say what happens to a partner that never answers.
4. You can explain what `Advance(249 ms)` followed by `Advance(1 ms)` proves in test 6.6, and why it cannot flake.
5. You can read the VB desk and name two things its C# twin does that VB cannot express.

🔍 Check yourself

1. An async method is called but its task is never awaited. Is the work running? How does that differ from a Reactor `Mono` nobody subscribes to?
2. Where does the code after an `await` run in ASP.NET Core, and why do libraries use `ConfigureAwait(false)` while application code does not?
3. `QueryOneAsync` catches `OperationCanceledException` only `when (!ct.IsCancellationRequested)`. What breaks without that filter when the customer leaves?
4. What does `Task.WhenEach` give you that `Task.WhenAll` does not, and which .NET version added it?
5. A bounded channel with `FullMode = Wait` has a slow consumer. What happens to the producer, and what would `DropOldest` change?
6. Why can a `lock` body not contain an `await`, and what do you use instead?
7. Name three async constructs the VB compiler rejects, and the C# form that compiles.

Lesson 05 in one line: a `Task` is hot, `await` resumes wherever the SynchronizationContext says, cancellation is a token you pass by hand — so fan out over per-call deadlines, stream through bounded channels, never block or lock across an `await`, and inject `TimeProvider`.
Next: lesson 06 (VB.NET for Legacy Estates) — reading, keeping and retiring the Visual Basic estates this lesson kept running into.